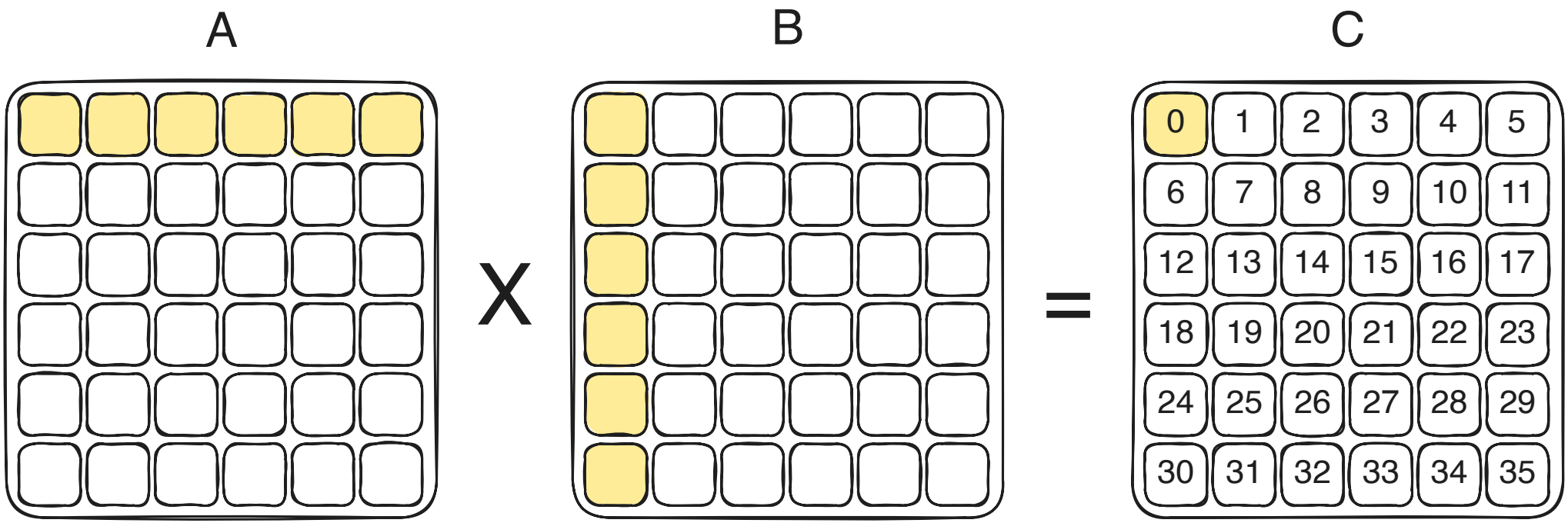
Tensor Contractions on GPUs

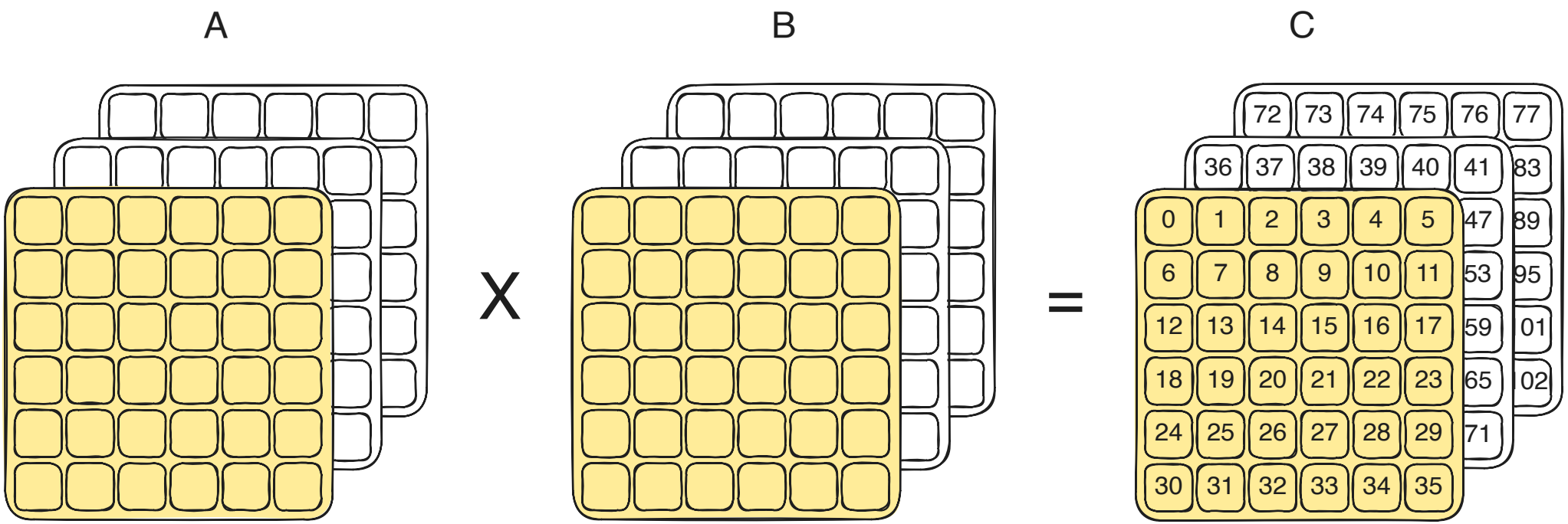
Max Koch

Recap: Tiled Matrix Multiplication on GPUs



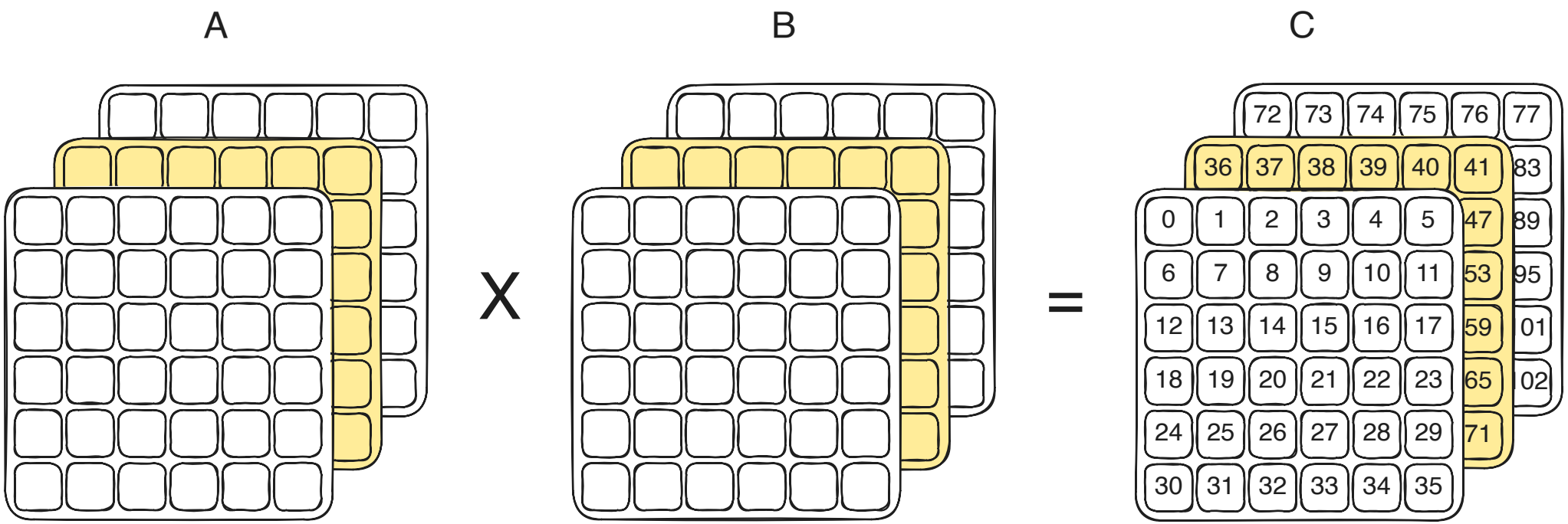
Batched Matrix Multiplication

- Multiple matrix multiplications along an additional batch dimension



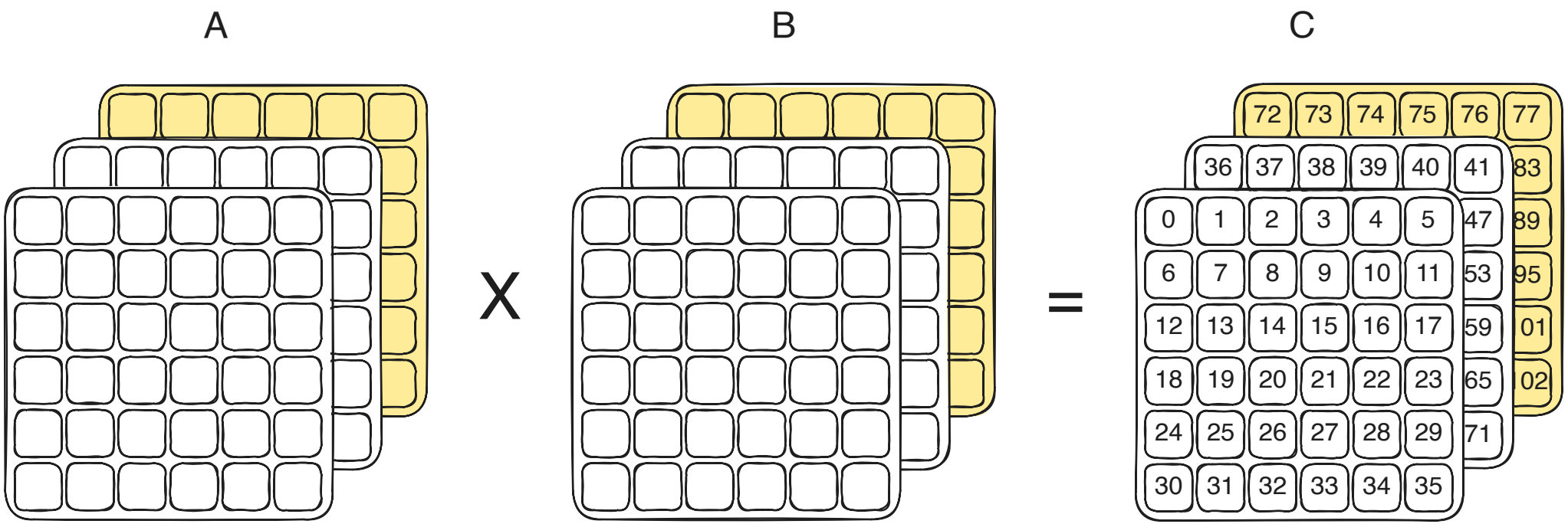
Batched Matrix Multiplication

- Multiple matrix multiplications along an additional batch dimension



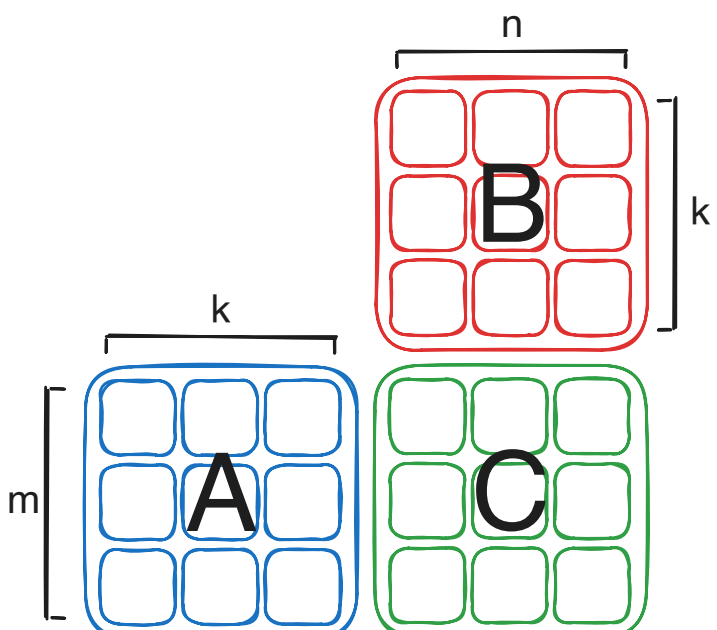
Batched Matrix Multiplication

- Multiple matrix multiplications along an additional batch dimension

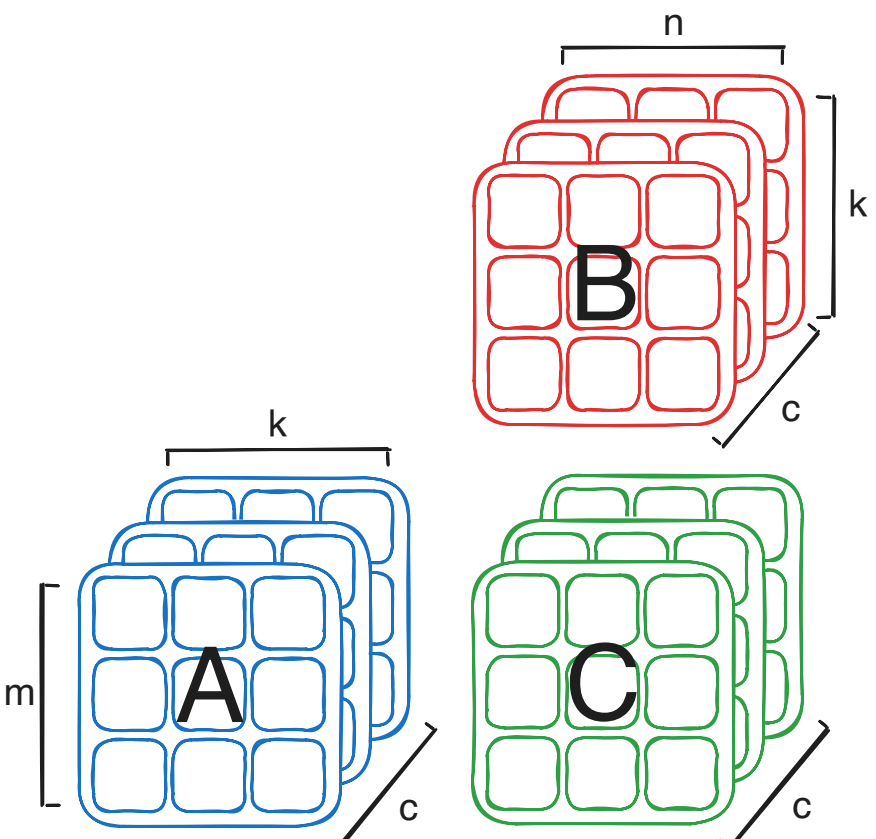


Recap: Einsum

- `C = torch.einsum("mk,kn->mn", A, B)`



- `C = torch.einsum("cmk,ckn->cmn", A, B)`



Computing Tensor Contractions on GPUs

Index Types in Einsum Expressions

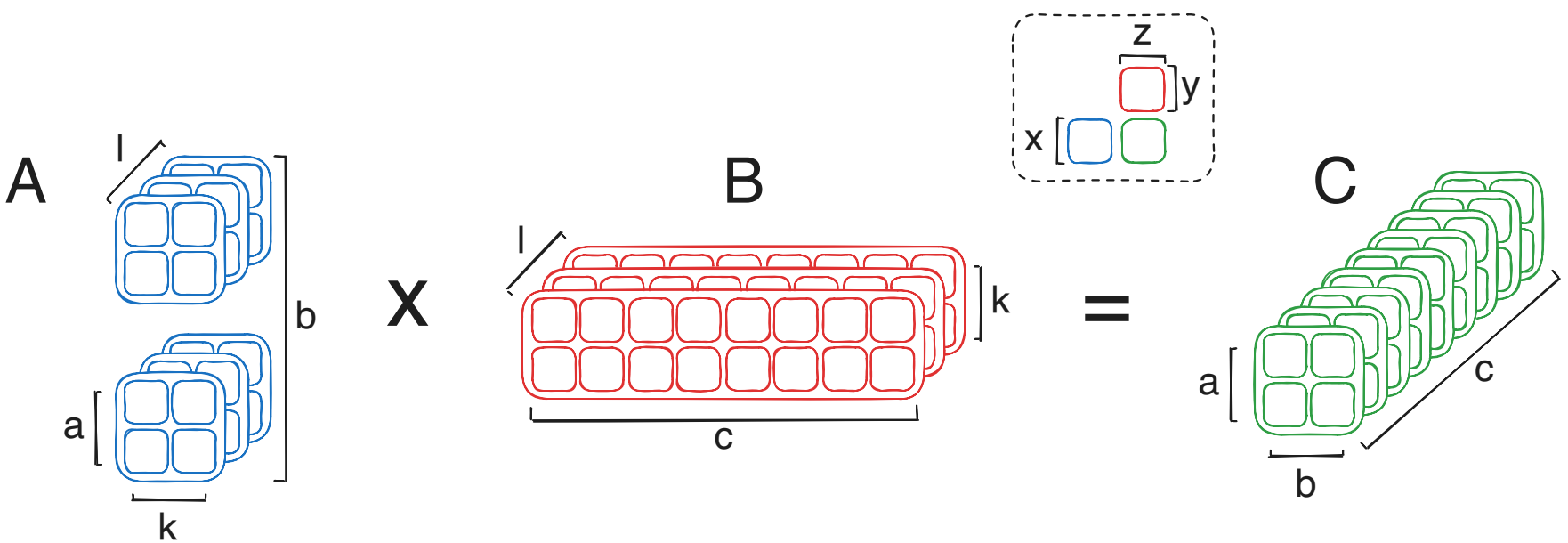
- Categorize indices of an einsum expression into *M* (free), *N* (free), *K* (contraction), and *C* (batch) indices

Type	Input Tensor A	Input Tensor B	Output Tensor C
M	✓		✓
N		✓	✓
K	✓	✓	
C	✓	✓	✓

Transpose-Transpose-GEMM-Transpose (TTGT)

- Transpose (permute) and reshape input tensors to view tensor contraction as a (batched) matrix multiplication
- After matrix multiplication, reshape and transpose the output tensor back to the desired layout

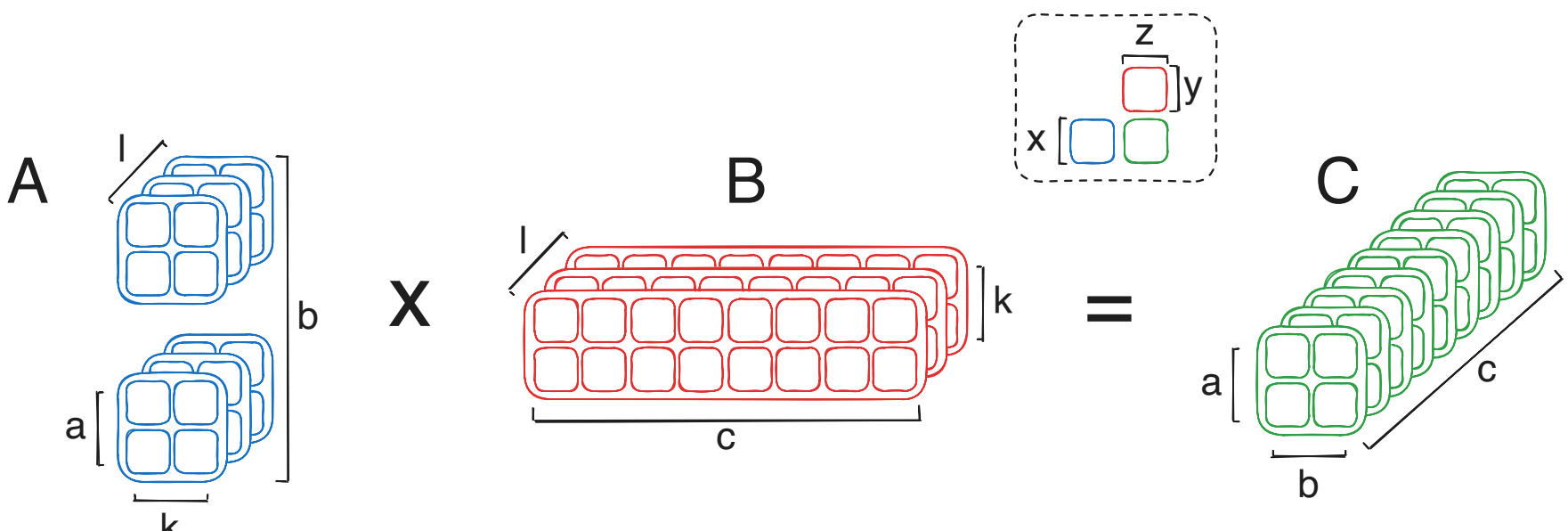
Example: `abklxy, cklyz -> abcxz`



Transpose-Transpose-GEMM-Transpose (TTGT)

- Transpose (permute) and reshape input tensors to view tensor contraction as a (batched) matrix multiplication
- After matrix multiplication, reshape and transpose the output tensor back to the desired layout

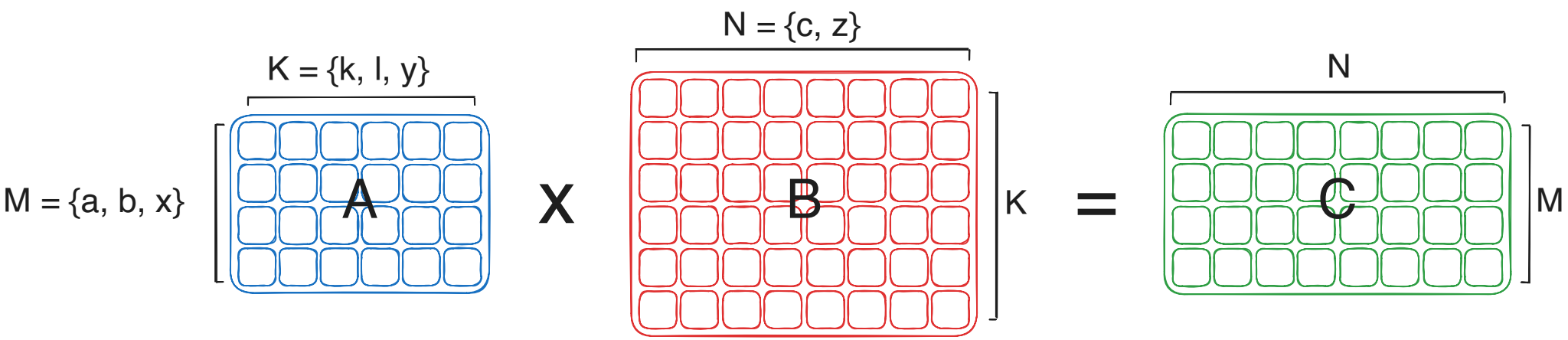
Example: `abklxy, cklyz -> abcxz` $M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$



Transpose-Transpose-GEMM-Transpose (TTGT)

- Transpose (permute) and reshape input tensors to view tensor contraction as a (batched) matrix multiplication
- After matrix multiplication, reshape and transpose the output tensor back to the desired layout

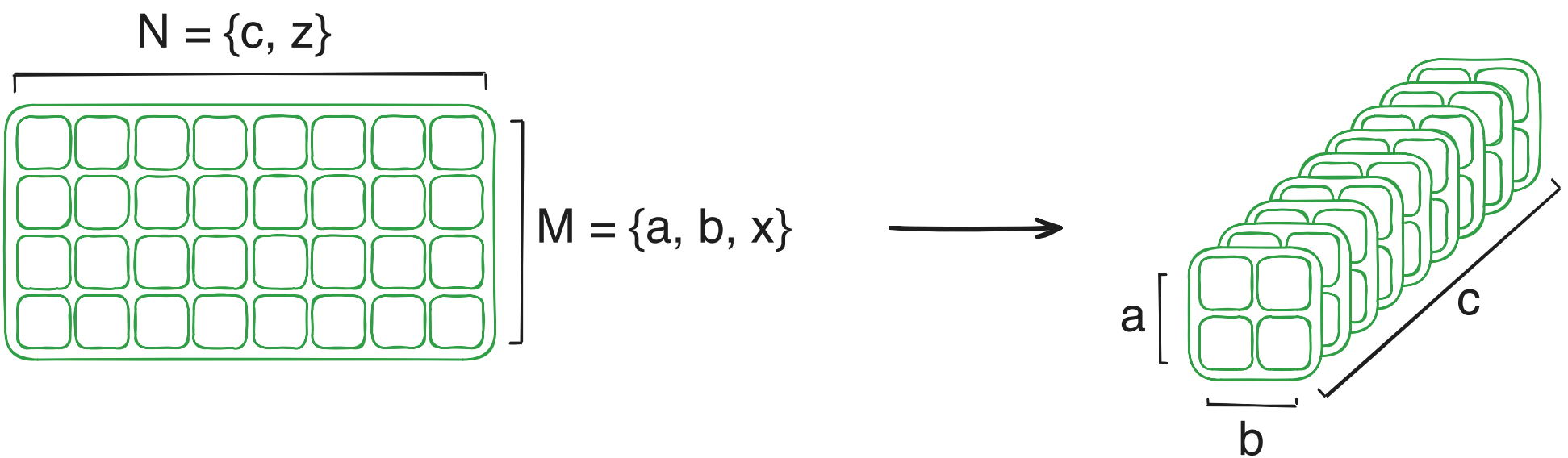
Example: `abklxy, cklyz -> abcxz` $M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$



Transpose-Transpose-GEMM-Transpose (TTGT)

- Transpose (permute) and reshape input tensors to view tensor contraction as a (batched) matrix multiplication
- After matrix multiplication, reshape and transpose the output tensor back to the desired layout

Example: `abklxy, cklyz -> abcxz` $M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$



Transpose-Transpose-GEMM-Transpose (TTGT)

Pros

- Leverages highly optimized GEMM kernels by reshaping tensors into 2D matrices
- TTGT handles arbitrary tensor shapes and contractions
- Low compilation overhead

Cons

- Potential global memory overhead because the transposed tensors have to be stored
-

Transpose-Transpose-GEMM-Transpose (TTGT)

Pros

- Leverages highly optimized GEMM kernels by reshaping tensors into 2D matrices
- TTGT handles arbitrary tensor shapes and contractions
- Low compilation overhead

Cons

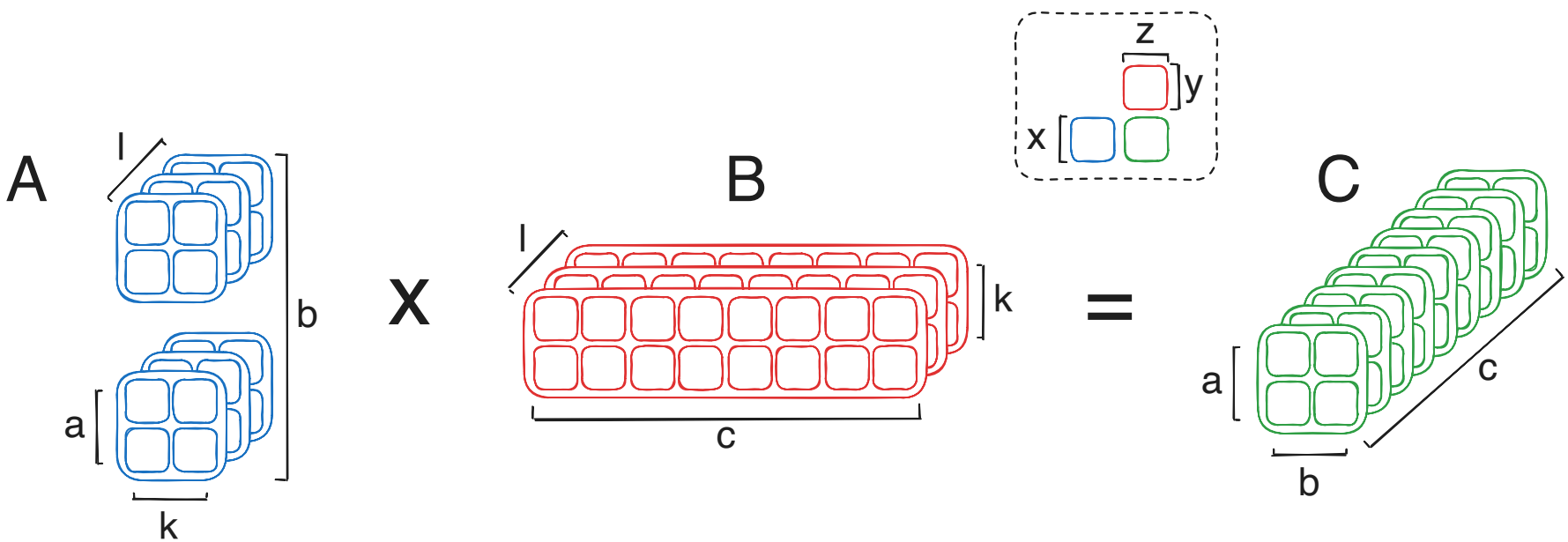
- Potential global memory overhead because the transposed tensors have to be stored
- **Permutation in memory is expensive**
  - For every contraction, three tensors have to be permuted in memory: both input tensors and the output tensor

Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz`

$$M = \{a, b, x\},\ N = \{c, z\},\ K = \{k, l, y\}$$

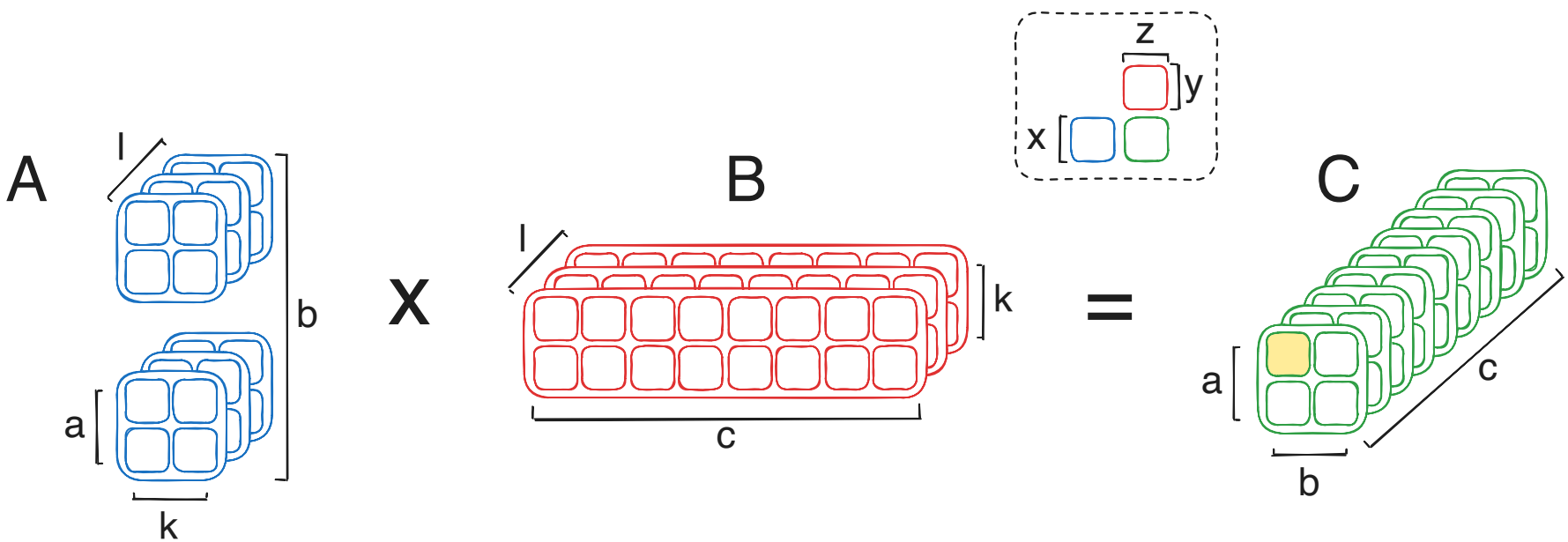


Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz`

$$M = \{a, b, x\},\ N = \{c, z\},\ K = \{k, l, y\}$$

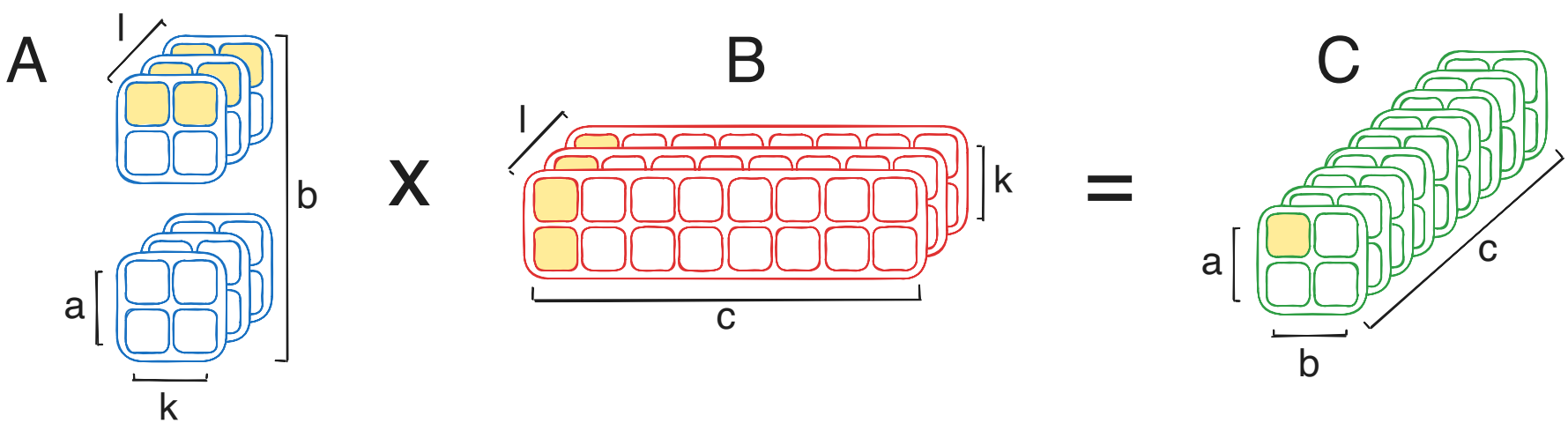


Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz`

$$M = \{a, b, x\},\ N = \{c, z\},\ K = \{k, l, y\}$$

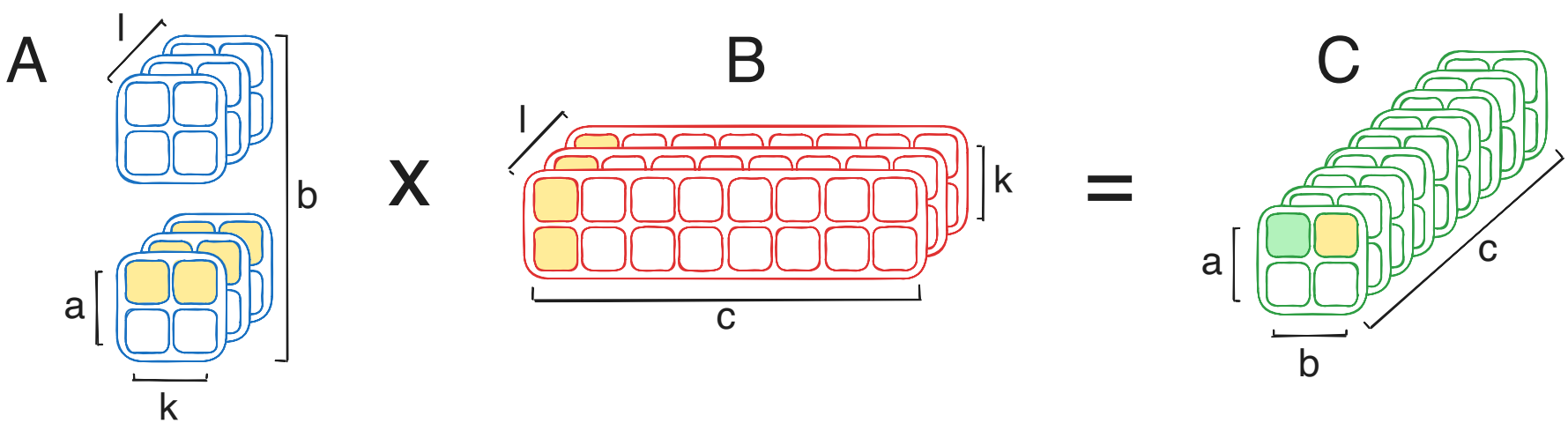


Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz`

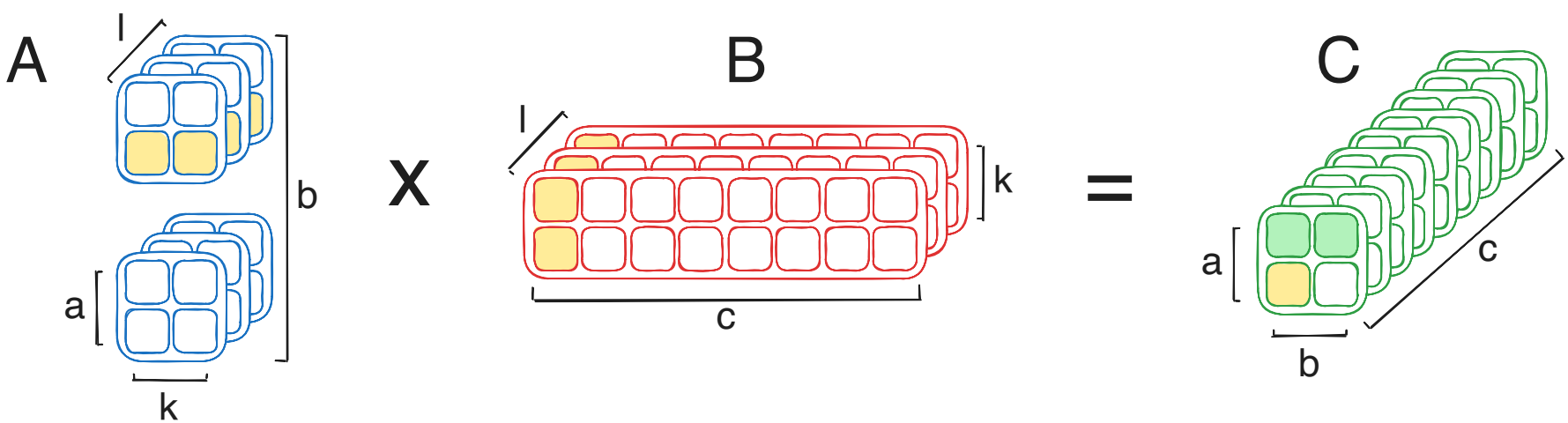
$$M = \{a, b, x\},\ N = \{c, z\},\ K = \{k, l, y\}$$



Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz` $M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$

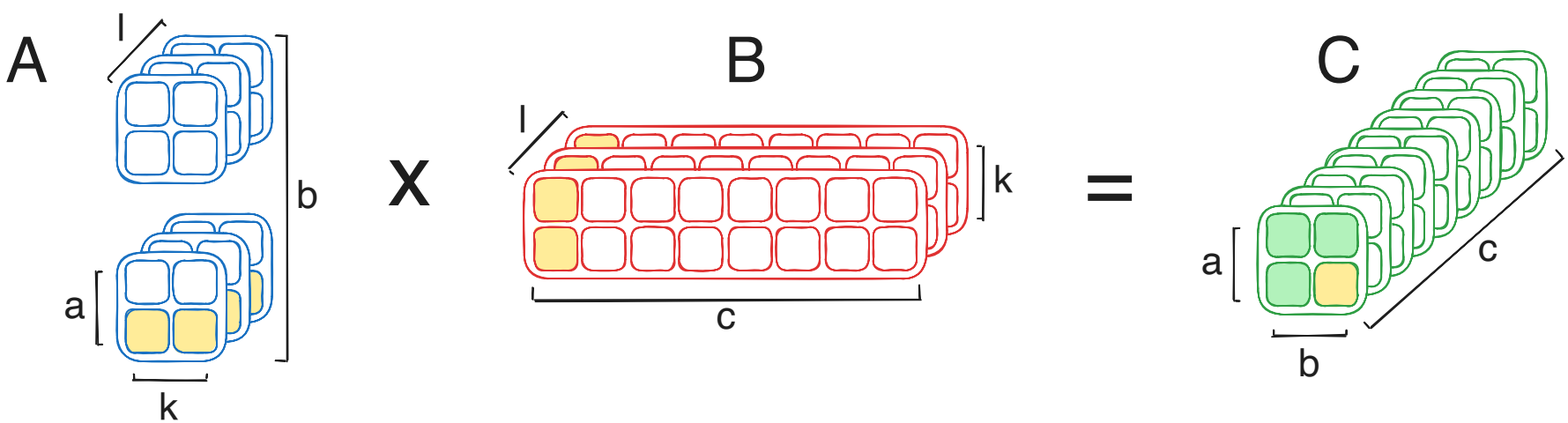


Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz`

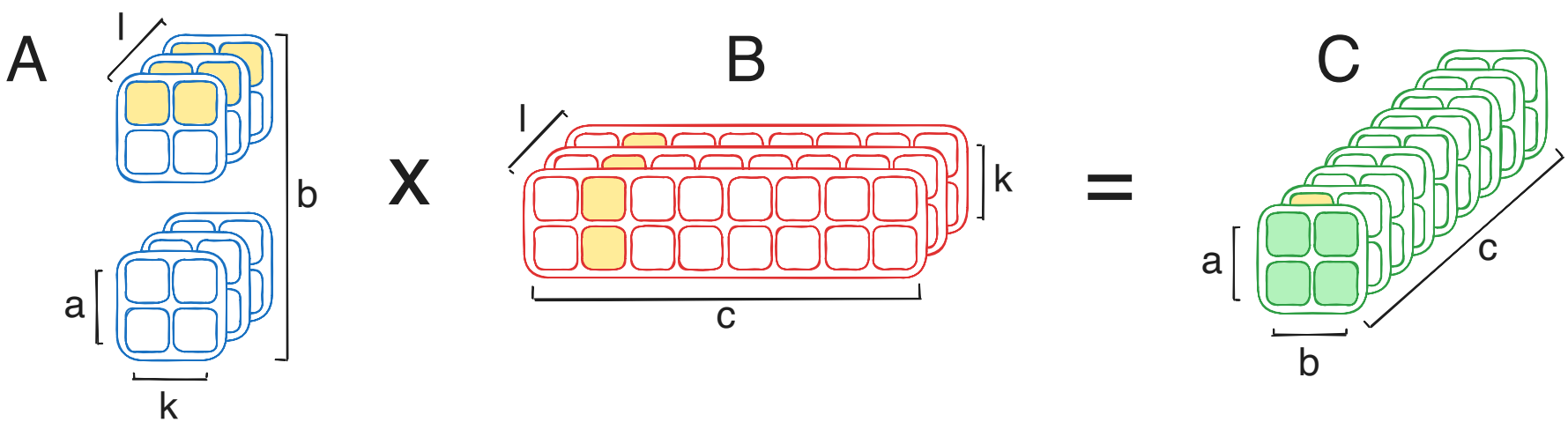
$M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$



Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz` $M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$

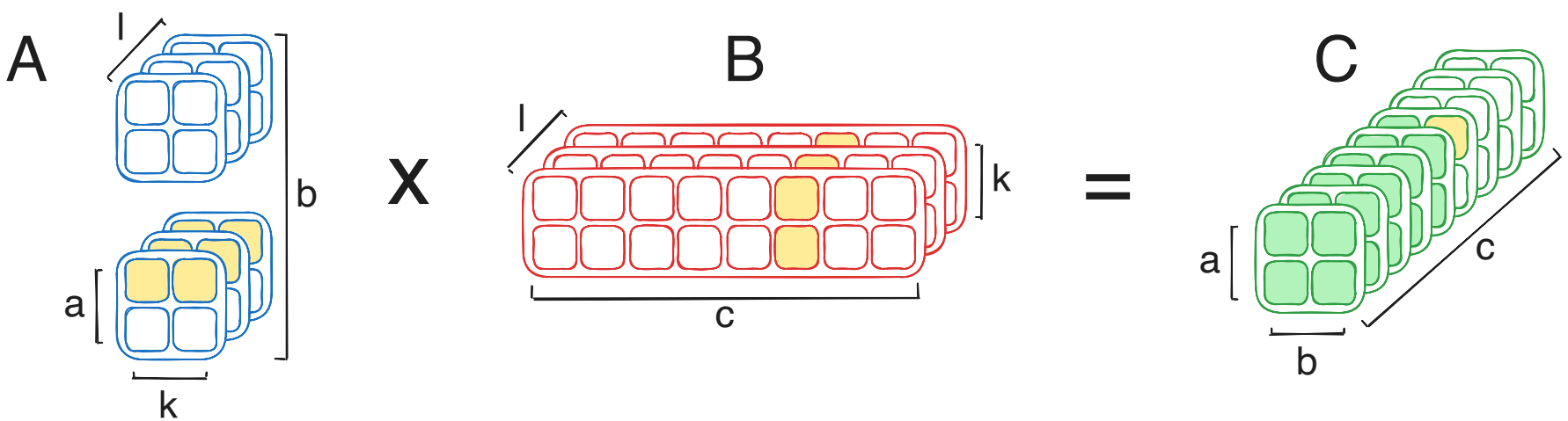


Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz`

$M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$

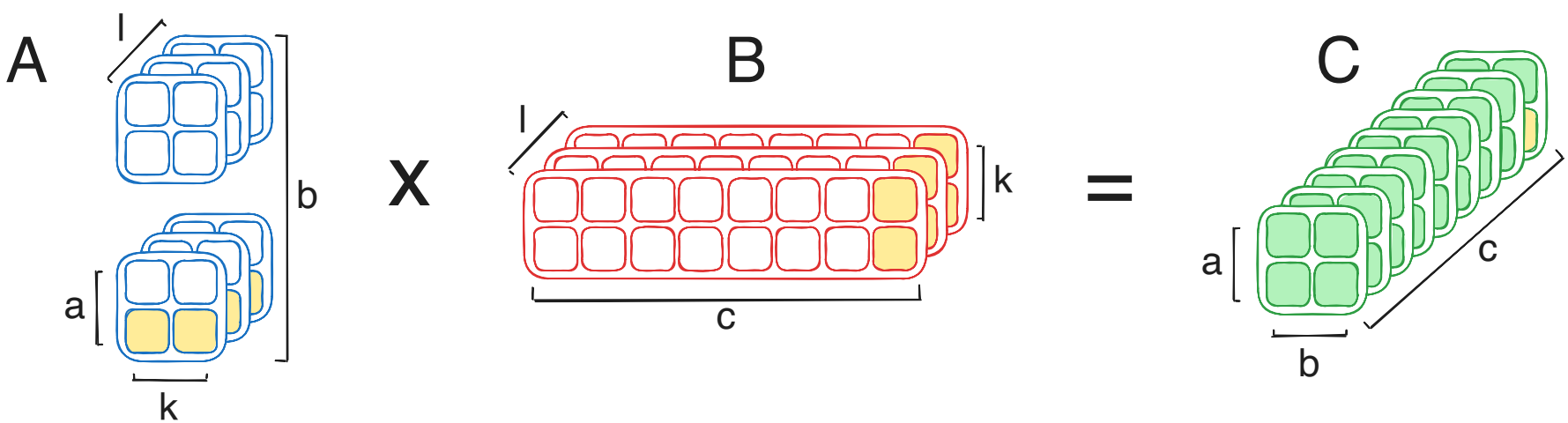


Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz`

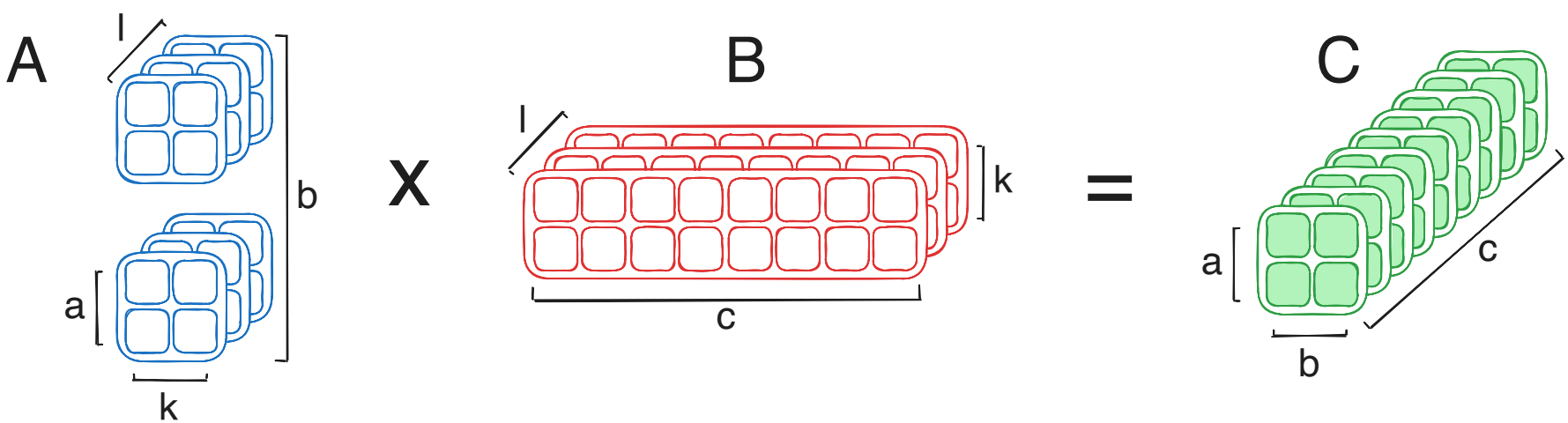
$M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$



Tiled Contraction

- Identify matrix multiplication tiles in the contraction with dimensions of type M, N and K
- Use other dimensions as grid dimensions or for sequential loops inside the kernel

Example: `abklxy, cklyz -> abcxz` $M = \{a, b, x\}$, $N = \{c, z\}$, $K = \{k, l, y\}$



cuTile Tensor Contraction

- Different implementations perform the same contraction: `abklxy, cklyz -> abcxz`

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (abc)

  for k_it in range(k):
    for l_it in range(l):
      # Matmul shape = (x, z, y)
```

cuTile Tensor Contraction

- Different implementations perform the same contraction: `abklxy, cklyz -> abcxz`

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (abc)

  for k_it in range(k):
    for l_it in range(l):
      # Matmul shape = (x, z, y)
```

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (bc)

  for a_it in range(a):
    for k_it in range(k):
      for l_it in range(l):
        # Matmul shape = (x, z, y)
```

cuTile Tensor Contraction

- Different implementations perform the same contraction: `abklxy, cklyz -> abcxz`

```python
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (abc)

  for k_it in range(k):
    for l_it in range(l):
      # Matmul shape = (x, z, y)
```

```python
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (abc)

  for k_it in range(k):
    # Matmul shape = (x, z, y * l)
```

```python
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (bc)

  for a_it in range(a):
    for k_it in range(k):
      for l_it in range(l):
        # Matmul shape = (x, z, y)
```

cuTile Tensor Contraction

- Different implementations perform the same contraction: `abklxy, cklyz -> abcxz`

```python
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (abc)

  for k_it in range(k):
    for l_it in range(l):
      # Matmul shape = (x, z, y)
```

```python
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (abc)

  for k_it in range(k):
    # Matmul shape = (x, z, y * l)
```

```python
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (bc)

  for a_it in range(a):
    for k_it in range(k):
      for l_it in range(l):
        # Matmul shape = (x, z, y)
```

```python
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid -> (abc)

  # Matmul shape = (x, z, y * l * k)
```

Kernel Fusion: Contraction and Elementwise Multiplication

No Fusion

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid
  # Loops
  for [...]():
    for [...]():
      ct.mma(...)

@ct.kernel
def elwise_multiplication(C, D):
  pid = ct.bid(0)
  # Decompose pid
  # Load tiles
  # Compute elwise multiplication
  # Store result

contraction(A, B, C)
elwise_multiplication(C, D)
```

Kernel Fusion: Contraction and Elementwise Multiplication

No Fusion

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # Decompose pid
  # Loops
  for [...]():
    for [...]():
      ct.mma(...)

@ct.kernel
def elwise_multiplication(C, D):
  pid = ct.bid(0)
  # Decompose pid
  # Load tiles
  # Compute elwise multiplication
  # Store result

contraction(A, B, C)
elwise_multiplication(C, D)
```

Fusion

```
@ct.kernel
def fused_kernel(A, B, C, D):
  pid = ct.bid(0)
  # Decompose pid
  # Loops
  for [...]():
    for [...]():
      ct.mma(...)
      # Compute elementwise multiplication


fused_kernel(A, B, C, D)
```